

The BIG Project - CS 396 - Pokemon Showdown

Daniel Raygoza

July 23, 2026

0.1 Architecture

As a pre-note, besides the three languages of **C++**, **Scheme** and **Prolog**, there is also the use of TypeScript for a 'Prebuilding Phase' which doesn't affect the main goal of the project and is just for gathering data and putting it into a usable format for both **C++** and **Prolog**. Nevertheless it will be covered as part of the architecture.

0.1.1 Components

Smogon_Scraper (TypeScript)

A small program that pulls JSON data from Smogon University(<https://www.smogon.com/dex/bw/>). This is neither an orchestrator nor is it called from the actual project, this is a prebuild program that generates one time files like the necessary database for **Prolog** to properly compute if a Pokemon is valid, and constants to put in a **cpp** file so that **C++** can do the proper damage calculations and track the PP of a move and such.

Disclaimer!

This project does use an internal library of mine that isn't bundled due to it's size although it can easily be installed if you follow the **README.md** within.

Pokemon Team Parser (Scheme)

This program will take in two text files as input, these text files should contain up to six Pokemon written in the format:

```
1 Zorua @ Life Orb
2 Level: 5
3 Ability: Illusion
4 EVs: 200 Atk / 40 Def / 240 Spe
5 IVs: 2 HP
6 Naive Nature
7 - Dark Pulse
8 - Sucker Punch
9 - Hidden Power Fighting
10 - Extrasensory
```

From these text files it will output two JSON array of objects that accurately describe the Pokemon so that both **C++** and **Prolog** can understand them.

Pokemon Team Validator (Prolog)

This program takes in JSON files containing a JSON array of objects that represent Pokemon and then from a prebuilt database validates each Pokemon to see if they could actually exist. This involves checking to see if Pokemon exists, their nickname is an acceptable length, the sum of their EVs don't go over the total EV threshold, if they have a hidden power move it must match their IVs based off a complicated set of rules based off which IVs are even and odd, if their moves in that Pokemon's movelist, if their item valid, if their nature valid, and if their ability valid. Once it checks a Pokemon, if it's valid it will continue to the next Pokemon, else it will immediately return a failure code. If it does make it to the end and all the Pokemon pass then it will just return a success code with no output.

Player / Orchestrator (C++)

This program initially acts as orchestrator and takes in two files each containing a Pokemon team for both the player and opponent. During the orchestrator phase it will feed the two files into the `Pokemon Team Parser` and if they both pass the parsing it will store the Pokemon teams in memory just in case the validating also passes. Then it will send over the output from the `Pokemon Team Parser` to the `Pokemon Team Validator`. If both teams pass validation then it will turn from `orchestrator` mode to `player` mode. `Player` mode is when the user actually get to perform a Pokemon battle against a `CPU`.

0.1.2 Bringing it together

So to summarize and bring it together the *successful* pipeline looks like:

1. `Smogon_Scraper` - Generate the necessary files for the `Pokemon Team Validator` and `Player` to be successful. [ONCE]
2. `Orchestrator` - Read 2 Pokemon team files and then pass them off to the `Pokemon Team Parser`.
3. `Pokemon Team Parser` - Read the Pokemon team files and output parsed JSON back to the `Orchestrator`.
4. `Orchestrator` - Send a copy of the parsed Pokemon team files to `Pokemon Team Validator`.
5. `Pokemon Team Validator` - Read the parsed Pokemon team files and validate them against a set of predefined rules in the `Prolog` database, then sending the result back to the `Orchestrator` that it succeeded.
6. `Orchestrator` → `Player`
7. `Player` - Allow the user to battle the computer with their valid teams.

0.2 Motivation

The motivation behind this project was mainly from my love of Pokemon, but also since I saw a good vision for how the architecture of the project could be handled between multiple languages and how object oriented programming could be used at the same time. Moreover, the original `Pokemon Showdown` itself may be open source but it's a large scale web based project that I felt could be interesting if something similar was running as a TUI on C++ which would likely be much faster. Next, unlike this project, the original `Pokemon Showdown` just uses `TypeScript` as a universal language rather than splitting it up based off of what tool is best for the job. Lastly, this project has the ability to play against a `CPU` which the original version doesn't seem to have.

0.3 Paradigm integration

The goal of this project is to create a system that involves `functional` abstraction in `Scheme`, `object-oriented design` in `C++`, and `logical reasoning` in `Prolog`.

Using `Scheme(Racket)`, which is a primarily a functional language allowed me to break down the parsing of `Showdown Export Format` into smaller and smaller components until the problem was effectively solved.

Most of the functions I wrote were very small and all the functions accomplished a single task. That was however until, I wanted some feedback of the general area of where a parsing error could have occurred and because of that besides just being a functional program I decided to add some global state. Now it's not much, it just stores the `last-line`, `pokemon-num`, and `current-file`. Going back to **functional abstraction**, when initially writing the program I would first just define many functions in the form of a **black box**. It contained just the name and parameters with no implementations. Then I would slowly sketch out inner functions using those functions until the whole program got mapped out.

Using **C++** I used both an imperative design for the **orchestrator mode**, simply running constant instructions that would have to be executed everytime the program would run. Fetching the necessary data from the user and then passing the data around between languages to get the final data to use for the actual player. No need for **object-oriented programming** is this phase. However, once the program switches from **orchestrator mode** to **player mode** then it must switch programming paradigms and in this case, over to **object-oriented programming**. Why? Well each Pokemon can have abilities that have different effects, with different conditions on how they can trigger and when they can trigger. Most Pokemon moves can cause an additional side effect. There are many different status effects a Pokemon can have that have different post-turn effects. You get the idea. In some languages, like **Python** or **TypeScript** you could easily write this as objects that store functions as values, however in **C++**, functions aren't first class citizens. So, this makes virtual functions and inheritance our best friend to design something like this. Each move, ability, item, and status all inherit a base version of themselves that have virtual functions that by default are **no-op** but can be implemented specifically to each one.

Disclaimer!

No abilities or items are implemented due to the insane scope of it and because it would be horrendous trying to read the code at that point, however using this design paradigm makes it fairly easy to implement the abilities and items. Moreover because of the lack of heavy implementation, most of these classes have a *base*, *base* class allowing me to store the constant data for these items which can be used anywhere and if I do implement the actual derived class then I can use a secondary table to fetch the base data so I don't interfere with the auto-generated tables. There are over 90 move effects implemented many of them making the game fairly playable with many teams. Just because the abilities and items aren't implemented for the player, does NOT mean they aren't all properly described in the **Prolog** database.

Lastly, using **Prolog** and it's **logical reasoning** based programming to validate whether or not the Pokemon teams were valid. With many **Prolog** programs the first thing that you'll need is your set of facts, and in my case was a 52000 line database of facts about all the Pokemon (although programmatically generated), that focused on which Pokemon exist, which moves exists, which items exists, which abilities exists, which natures exists, which Pokemon have what moves, and which Pokemon have what abilities. This wasn't my only database of facts however, since as mentioned earlier there is a special move that has it's type determined by the Pokemons IVs. My second database of facts simply manually labeled the variations of the move **Hidden Power** as both a move and as **Hidden Power**. With all the facts we can now validate our data by then breaking down the end validation into much smaller and digestible predicates, such as: `pokemon_move_valid`, `pokemon_all_moves_valid`, `pokemon_hidden_power_valid`, `pokemon_nickname_valid`, `pokemon_evs_valid`, and more helpers.